Digital Logic Design Laboratory

Logic Circuit Analysis Using NAND Gates

Arduino Implementation using Push Buttons and LED

Submitted by:
Rugved Kshirsagar

July 2, 2026

Department of Electronics / Embedded Systems

Digital Logic Design Laboratory

Experiment Report

Academic Year 2025–26

Contents

1 Objective

The objectives of this experiment are:

- To analyze the given digital logic circuit.
- To derive the Boolean expression corresponding to the circuit.
- To simplify the expression using Boolean algebra and De Morgan's theorem.
- To verify the derived expression using a truth table.
- To implement the logic function using an Arduino Uno.
- To understand how digital logic can be realized using software and hardware.

2 Introduction

Digital logic forms the foundation of modern electronic systems. Every digital device, ranging from calculators and smartphones to computers and industrial controllers, operates by processing binary signals represented by logic HIGH (1) and logic LOW (0).

Logic gates are the basic building blocks of digital circuits. Each gate performs a specific Boolean operation on one or more binary inputs to produce a binary output.

Common logic gates include:

- AND Gate
- OR Gate
- NOT Gate
- NAND Gate
- NOR Gate
- XOR Gate
- XNOR Gate

Among these, the NAND gate is of particular importance because it is a universal gate. Any Boolean function can be implemented exclusively using NAND gates.

3 Theory

3.1 Boolean Algebra

Boolean algebra is a branch of mathematics that deals with binary variables and logical operations. Unlike ordinary algebra, Boolean variables can take only two values:

$$0 \quad \text{(Logic LOW)}$$

and

1 (Logic HIGH)

The fundamental Boolean operations are:

- AND ($\cdot$)
- OR ($+$)
- NOT ($\overline{\ }$)

3.2 The NAND Gate

A NAND gate is the complement of an AND gate.

For two inputs A and B, the output is

$$Y = \overline{AB}$$

Its truth table is shown below.

A	B	$Y = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

3.3 Universal Gate

A NAND gate is called a universal gate because every other logic gate can be realized using only NAND gates.

For example,

$$\text{NOT}(A) = \overline{AA}$$

$$A \cdot B = \overline{\overline{AB}}$$

$$A + B = \overline{\overline{A}\,\overline{B}}$$

Therefore, a complete digital circuit can be constructed entirely from NAND gates.

3.4 De Morgan's Theorem

De Morgan's theorem is one of the most important identities in Boolean algebra.

The two identities are

$$\overline{A + B} = \overline{A}\,\overline{B}$$

and

$$\overline{AB} = \overline{A} + \overline{B}$$

These identities are extensively used for simplifying digital logic circuits, especially circuits constructed using NAND and NOR gates.

3.5 Method of Analysis

The analysis of logic circuits generally follows these steps:

1. Identify every logic gate in the circuit.
2. Write the Boolean expression for the output of each gate.
3. Substitute intermediate expressions into the final expression.
4. Apply Boolean algebra and De Morgan's theorem.
5. Obtain the simplified Boolean expression.
6. Verify the result using a truth table.

4 Problem Statement

The logic circuit shown in Figure 1 consists entirely of NAND gates. Determine the Boolean expression for the output Y.

38. In the logic circuit shown in Fig. 3.21, Y is given by

a) $Y = ABCD$
b) $Y = (A + B)(C + D)$
c) $Y = A + B + C + D$
d) $Y = AB + CD$

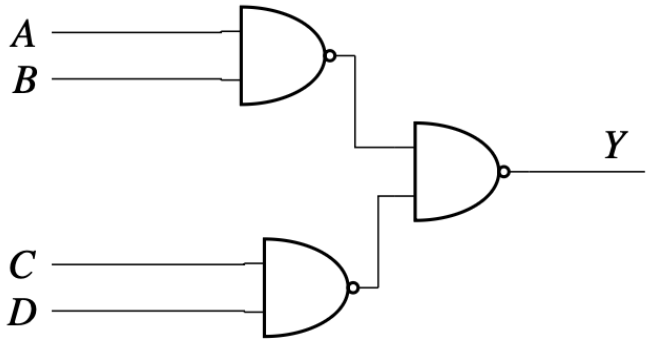

Fig. 3.21

Figure 1: Logic circuit consisting of NAND gates (GATE EE 2018).

The available options are

$$(a)\ Y = ABCD$$
$$(b)\ Y = (A + B)(C + D)$$
$$(c)\ Y = A + B + C + D$$
$$(d)\ Y = AB + CD$$

5 Circuit Analysis

The circuit contains three NAND gates. The outputs of the first two NAND gates are connected to the inputs of the third NAND gate.

5.1 Step 1: Output of the First NAND Gate

The first NAND gate receives inputs A and B.

Therefore,

$$X_1 = \overline{AB}$$

5.2 Step 2: Output of the Second NAND Gate

Similarly, the second NAND gate receives inputs C and D.

Hence,

$$X_2 = \overline{CD}$$

5.3 Step 3: Output of the Final NAND Gate

The outputs of the first two NAND gates become the inputs of the final NAND gate.

Therefore,

$$Y = \overline{X_1 X_2}$$

Substituting the intermediate expressions,

$$Y = \overline{\left(\overline{AB}\right)\left(\overline{CD}\right)}$$

5.4 Step 4: Applying De Morgan's Theorem

Using the identity

$$\overline{PQ} = \overline{P} + \overline{Q},$$

where

$$P = \overline{AB}$$

and

$$Q = \overline{CD},$$

we obtain

$$Y = \overline{\overline{AB}} + \overline{\overline{CD}}$$

Since a double complement cancels,

$$\overline{\overline{AB}} = AB$$

and

$$\overline{\overline{CD}} = CD,$$

therefore,

$$Y = AB + CD$$

Hence,

$$\boxed{Y = AB + CD}$$

Therefore, the correct option is

$$\boxed{\textbf{Option (d)}}$$

5.5 Discussion

The first two NAND gates generate the complements of the products AB and CD. The third NAND gate complements the product of these intermediate outputs. By applying De Morgan's theorem, the final expression simplifies to the logical OR of the two products, namely

$$Y = AB + CD.$$

This demonstrates how a network of NAND gates can implement logic functions that are not immediately obvious from the circuit diagram.

6 Truth Table Verification

To verify the derived Boolean expression, all possible combinations of the four input variables are considered.

The Boolean expression obtained from the circuit analysis is

begin : *math* : *display* Y ĀB CD *end* : *math* : *display*

The intermediate terms *begin* : *math* : *text*AB*end* : *math* : *text* and *begin* : *math* : *text*CD*end* : *math* : *text* are first evaluated, after which the logical OR operation is performed to obtain the final output.

Table 1: Truth table for the logic circuit

A	B	C	D	AB	CD	Y=AB+CD
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	0
0	0	1	1	0	1	1
0	1	0	0	0	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	1	1	0	1	1
1	0	0	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	1
1	1	0	1	1	0	1
1	1	1	0	1	0	1
1	1	1	1	1	1	1

6.1 Verification

From the truth table, the following observations can be made:

1. The output becomes HIGH whenever both inputs *begin* : *math* : *text*A*end* : *math* : *text* and *begin* : *math* : *text*B*end* : *math* : *text* are HIGH.
2. The output also becomes HIGH whenever both inputs *begin* : *math* : *text*C*end* : *math* : *text* and *begin* : *math* : *text*D*end* : *math* : *text* are HIGH.
3. If neither pair satisfies the AND condition, the output remains LOW.
4. The truth table exactly matches the Boolean expression *begin* : *math* : *display* Y ĀB CDend:math:display

Hence, the analytical solution obtained using Boolean algebra is verified.

6.2 Result

The derived Boolean expression,

$begin : math : display$

boxed{Y $\bar{A}$B CD} $end : math : display$

is therefore correct and corresponds to Option (d).

7 Arduino Implementation

The derived Boolean expression was implemented using an Arduino Uno to demonstrate the practical realization of the logic circuit. Four push buttons were used as digital inputs representing the variables $begin : math : textAend : math : text$, $begin : math : textBend : math : text$, $begin : math : textCend : math : text$, and $begin : math : textDend : math : text$. An LED connected to one of the digital output pins was used to display the output of the Boolean function.

Instead of constructing the circuit using physical NAND ICs, the logic was implemented in software using the Arduino microcontroller. This approach simplifies testing while preserving the functionality of the original digital circuit.

7.1 Hardware Components

The following components were used during the implementation.

Table 2: List of Hardware Components

Component	Quantity
Arduino Uno	1
Breadboard	1
Push Buttons	4
LED	1
220 Ω Resistor	1
Jumper Wires	As Required
USB Cable	1

7.2 Pin Configuration

The digital pins of the Arduino Uno were assigned as shown in Table 3.

Table 3: Arduino Pin Configuration

Signal	Arduino Pin
Input A	D2
Input B	D3
Input C	D4
Input D	D5
Output LED	D13

7.3 Circuit Connections

The circuit connections are summarized below.

1. Connect one terminal of Push Button A to digital pin D2.
2. Connect one terminal of Push Button B to digital pin D3.

3. Connect one terminal of Push Button C to digital pin D4.
4. Connect one terminal of Push Button D to digital pin D5.
5. Connect the remaining terminal of every push button to Ground (GND).
6. Configure all four pins using the Arduino's internal pull-up resistors.
7. Connect the anode of the LED to digital pin D13 through a 220 Ω resistor.
8. Connect the cathode of the LED to Ground.

7.4 Working Principle

When a push button is pressed, the corresponding input becomes logic HIGH after software inversion because the Arduino's internal pull-up resistors are used.

The Arduino continuously performs the following sequence:

1. Read the four input pins.
2. Convert the active-low button signals into logic HIGH or LOW.
3. Compute the Boolean expression
   $begin : math : display$ Y
3. If the value of $begin : math : text$Y$end : math : text$ is equal to 1, turn ON the LED.
4. Otherwise, keep the LED OFF.
5. Repeat the process continuously.

7.5 Advantages of Software Implementation

Implementing the logic circuit using an Arduino provides several advantages.

- Eliminates the need for multiple logic ICs.
- Easy to modify the Boolean expression through software.
- Enables rapid testing of different logic circuits.
- Simplifies debugging.
- Demonstrates the relationship between digital hardware and embedded programming.

8 Arduino Program (Embedded C)

The Boolean expression obtained from the logic circuit,

$begin : math : display$ Y $\bar{A}B$ CD $end : math : display$

was implemented using the Arduino programming language (Embedded C). The Arduino continuously reads the state of four push buttons connected to the digital input pins and evaluates the Boolean expression. Depending on the result, the onboard LED connected to digital pin D13 is turned ON or OFF.

8.1 Arduino Source Code

```
1
2  const int A = 2;
3  const int B = 3;
4  const int C = 4;
5  const int D = 5;
6
7  const int LED = 13;
8
9  void setup()
10 {
11     pinMode(A, INPUT_PULLUP);
12     pinMode(B, INPUT_PULLUP);
13     pinMode(C, INPUT_PULLUP);
14     pinMode(D, INPUT_PULLUP);
15
16     pinMode(LED, OUTPUT);
17 }
18
19 void loop()
20 {
21     bool a = !digitalRead(A);
22     bool b = !digitalRead(B);
23     bool c = !digitalRead(C);
24     bool d = !digitalRead(D);
25
26     bool y = (a && b) || (c && d);
27
28     digitalWrite(LED, y);
29
30     delay(20);
31 }
```

Listing 1: Arduino Program for Implementing the Boolean Function

8.2 Program Explanation

The operation of the program is explained below.

1. Digital pins D2, D3, D4 and D5 are configured as input pins using the Arduino's internal pull-up resistors.

2. Digital pin D13 is configured as an output pin to drive the LED.
3. The Arduino continuously reads the state of the four push buttons.
4. Since INPUT_PULLUP is used, a pressed button reads as LOW. Therefore, the input values are inverted using the logical NOT operator.
5. The Boolean expression

   $begin : math : display$ Y
5. If the output evaluates to logic HIGH, the LED is turned ON.
6. Otherwise, the LED remains OFF.
7. The loop repeats continuously, allowing real-time operation of the digital logic circuit.

8.3 Algorithm

The following algorithm summarizes the operation of the Arduino program.

1. Start the program.
2. Configure digital pins D2–D5 as inputs.
3. Configure digital pin D13 as output.
4. Read the four input signals.
5. Convert active-low inputs into logic values.
6. Evaluate

   $begin : math : display$ Y ĀB CDend:math:display
7. If $begin : math : text$Y$\bar{1}end : math : text$, switch ON the LED.
8. Otherwise, switch OFF the LED.
9. Repeat Steps 4–8 indefinitely.

8.4 Expected Output

The LED behaves according to the Boolean expression.

- The LED turns ON whenever both A and B are HIGH.
- The LED also turns ON whenever both C and D are HIGH.
- The LED remains OFF for all other input combinations.

Thus, the Arduino successfully emulates the given digital logic circuit using software.

9 AVR Assembly Implementation

Although the Arduino implementation was written in Embedded C, the same Boolean function can also be implemented directly in AVR Assembly language. Assembly programming provides greater control over the hardware and executes with minimal overhead, making it suitable for low-level embedded applications.

The assembly program presented below targets the ATmega328P microcontroller used on the Arduino Uno.

9.1 AVR Assembly Program

```
1
2  .include "m328Pdef.inc"
3
4  .org 0x0000
5  rjmp MAIN
6
7  MAIN:
8
9      ; Configure PB5 (Arduino D13) as output
10     sbi DDRB,5
11
12 LOOP:
13
14     ; Read Port D
15     in r16,PIND
16
17     ; Buttons use INPUT_PULLUP
18     ; Therefore invert the bits
19     com r16
20
21     ; Check A AND B
22     mov r17,r16
23     andi r17,0b00001100
24     cpi r17,0b00001100
25     breq LED_ON
26
27     ; Check C AND D
28     mov r17,r16
29     andi r17,0b00110000
30     cpi r17,0b00110000
31     breq LED_ON
32
33 LED_OFF:
34
35     cbi PORTB,5
36     rjmp LOOP
37
38 LED_ON:
39
```

```
40      sbi  PORTB,5
41      rjmp LOOP
```

Listing 2: AVR Assembly Program for Implementing the Boolean Function

9.2 Explanation of the Program

The operation of the assembly program is summarized below.

1. The microcontroller starts execution from the reset vector.
2. Pin PB5 (Arduino pin D13) is configured as an output.
3. The program continuously reads the status of Port D.
4. Since the push buttons use internal pull-up resistors, the input bits are inverted using the `COM` instruction.
5. The program checks whether both inputs A and B are HIGH.
6. If this condition is satisfied, the LED is turned ON.
7. Otherwise, the program checks whether both inputs C and D are HIGH.
8. If either condition is true, the LED is switched ON.
9. If neither condition is satisfied, the LED is switched OFF.
10. The program then repeats indefinitely.

9.3 Registers Used

Table 4: Registers Used in the Assembly Program

Register	Purpose
R16	Stores the input data read from Port D
R17	Temporary register for logical operations
DDRB	Configures Port B direction
PORTB	Controls the LED output
PIND	Reads the input buttons

9.4 Comparison Between Embedded C and Assembly

Table 5: Comparison of Embedded C and AVR Assembly

Embedded C	AVR Assembly
Easy to understand	Difficult to learn
Portable between controllers	Device specific
Faster development	Longer development time
Compiler generates machine code	Programmer writes machine instructions
Easier debugging	More difficult debugging
Suitable for most applications	Suitable for highly optimized applications

The Embedded C implementation is preferred for rapid development, whereas AVR Assembly provides greater control over hardware resources and execution speed.

10 Program Flowchart

The flowchart shown below summarizes the operation of the Arduino program.

11 Experimental Observations

During the experiment, the output LED was observed for different combinations of the input push buttons.

Table 6: Sample Experimental Observations

A	B	C	D	LED State
0	0	0	0	OFF
1	1	0	0	ON
0	0	1	1	ON
1	1	1	0	ON
1	0	1	1	ON
1	1	1	1	ON

The observations matched the expected truth table for the Boolean expression $begin : math : display$ YĀBCDend:math:display

12 Advantages

The implementation presented in this experiment offers the following advantages.

- Simple realization of Boolean expressions.
- Easy hardware verification using LEDs.
- Faster than constructing multiple logic gate IC circuits.
- Easily modifiable through software.
- Suitable for educational demonstrations.
- Helps understand the relationship between digital electronics and embedded systems.

13 Applications

The concepts demonstrated in this experiment have applications in several fields, including

- Digital control systems

- Industrial automation
- Embedded systems
- Robotics
- Security systems
- Programmable Logic Controllers (PLCs)
- Digital signal processing
- Computer architecture

14 Conclusion

The given digital logic circuit was successfully analyzed using Boolean algebra. By identifying the outputs of each NAND gate and applying De Morgan's theorem, the Boolean expression

$begin : math : display$ Y$\bar{A}$BCD $end : math : display$

was obtained.

The derived expression was verified using a complete truth table. The same Boolean function was then implemented on an Arduino Uno using four push buttons as digital inputs and an LED as the output indicator. The Embedded C program correctly evaluated the Boolean expression in real time, while the AVR Assembly implementation demonstrated how the same logic could be realized at the machine level.

The experimental observations agreed with the theoretical analysis, thereby validating the correctness of the circuit and the software implementation.

15 References

1. M. Morris Mano and Michael D. Ciletti, *Digital Design*, Pearson.
2. Thomas L. Floyd, *Digital Fundamentals*, Pearson.
3. Arduino Documentation, `https://www.arduino.cc/`
4. ATmega328P Datasheet, Microchip Technology.
5. GATE EE 2018 Previous Year Question Paper.